

lab02

AUTHOR

Otavio Ribeiro Mello

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada ($\text{ARRIVAL_DELAY} > 10$)?

As estatísticas suficientes para determinar o percentual de atrasos em lotes (chunks) são: 1. $n_atrasos$: Número total de voos com atraso na chegada > 10 min (soma de $I(\text{ARRIVAL_DELAY} > 10)$). 2. n_total : Número total de voos válidos no grupo (contagem N).

2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):

a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;

```
# Bloco ilustrativo: 'input' representa um lote genérico de dados (um chunk)
# e só existe de fato dentro da função getStats() (item "e", abaixo). Por isso
# este bloco fica com eval: false -- ele documenta o passo, mas não é executado
# isoladamente (senão o render pararia com "object 'input' not found").
df_a <- input %>%
  filter(AIRLINE %in% c("AA", "DL", "UA", "US"))
```

b. Remova observações que tenham valores faltantes em campos de interesse;

```
df_b <- df_a %>%
  filter(!is.na(MONTH), !is.na(DAY), !is.na(AIRLINE), !is.na(ARRIVAL_DELAY))
```

c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;

```
df_c <- df_b %>%
  group_by(MONTH, DAY, AIRLINE)
```

d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe tibble;

```
df_d <- df_c %>%
  summarise(
    n_atrasos = sum(ARRIVAL_DELAY > 10),
    n_total = n(),
    .groups = "drop"
  ) %>%
  as_tibble()
```

e. A função deve receber apenas dois argumentos: `input`: o conjunto de dados (referente ao lote em questão); `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```

getStats <- function(input, pos) {
  # Passo a
  df_a <- input %>%
    filter(AIRLINE %in% c("AA", "DL", "UA", "US"))

  # Passo b
  df_b <- df_a %>%
    filter(!is.na(MONTH), !is.na(DAY), !is.na(AIRLINE), !is.na(ARRIVAL_DELAY))

  # Passo c
  df_c <- df_b %>%
    group_by(MONTH, DAY, AIRLINE)

  # Passo d
  df_d <- df_c %>%
    summarise(
      n_atrasos = sum(ARRIVAL_DELAY > 10),
      n_total = n(),
      .groups = "drop"
    ) %>%
    as_tibble()

  return(df_d)
}

```

3. Utilize alguma função readr::read_***_chunked para importar o arquivo flights.csv.zip. a. Configure o tamanho do lote (chunk) para 100 mil registros;

```
chunk_size <- 100000
```

b. Configure a função de callback para instanciar DataFrames aplicando a função getStats criada acima;

```
cb <- DataFrameCallback$new(getStats)
```

c. Configure o argumento col_types de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```

col_spec <- cols_only(
  MONTH = col_integer(),
  DAY = col_integer(),
  AIRLINE = col_character(),
  ARRIVAL_DELAY = col_double()
)

# Execução da leitura por lotes
stats_chunks <- read_csv_chunked(
  file = "flights.csv.zip",
  callback = cb,
  chunk_size = chunk_size,
  col_types = col_spec
)

```

4.Crie uma função chamada computeStats que: a.Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);

```
df_metrica <- stats_chunks %>%
  group_by(AIRLINE, MONTH, DAY) %>%
  summarise(
    total_atrasos = sum(n_atrasos),
    total_voos = sum(n_total),
    .groups = "drop"
  ) %>%
  mutate(
    Perc = total_atrasos / total_voos,
    Data = as.Date(paste(2015, MONTH, DAY, sep = "-"))
  )
```

b.Retorne as informações em um tibble contendo apenas as seguintes colunas: Cia: sigla da companhia aérea; Data: data, no formato AAAA-MM-DD (dica: utilize o comando as.Date); Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo [0,1].

Definição oficial da função computeStats unindo os passos a e b:

```
computeStats <- function(df) {
  res <- df %>%
    group_by(AIRLINE, MONTH, DAY) %>%
    summarise(
      total_atrasos = sum(n_atrasos),
      total_voos = sum(n_total),
      .groups = "drop"
    ) %>%
    mutate(
      Perc = total_atrasos / total_voos,
      Data = as.Date(paste(2015, MONTH, DAY, sep = "-"))
    ) %>%
    select(
      Cia = AIRLINE,
      Data,
      Perc
    )

  return(res)
}
```

Executa a função e gera o objeto 'stats_final' usado na questão 5:

```
stats_final <- computeStats(stats_chunks)
```

5.Produza um mapa de calor em formato de calendário para cada Cia. Aérea. a.Instale e carregue os pacotes ggcal e ggplot2.

```
# Só instala se ainda não estiver instalado, para não reinstalar a cada render
if (!requireNamespace("devtools", quietly = TRUE)) install.packages("devtools")
if (!requireNamespace("ggcal", quietly = TRUE)) devtools::install_github("jayjacobs/ggcal")
```

```
library(ggplot2)
library(ggcal)
```

b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.

```
pal <- scale_fill_gradient(
  low = "#4575b4",
  high = "#d73027"
)
```

c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá: Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`. Retornar para o usuário a base do calendário criada acima.

```
baseCalendario <- function(stats, cia) {
  # Criar um subconjunto de stats para a Cia. Aérea de interesse
  df_sub <- stats %>%
    filter(Cia == cia)

  # Montar a base do calendário com ggcal(x, y)
  cal <- ggcal(df_sub$Data, df_sub$Perc)

  # Retornar a base do calendário
  return(cal)
}
```

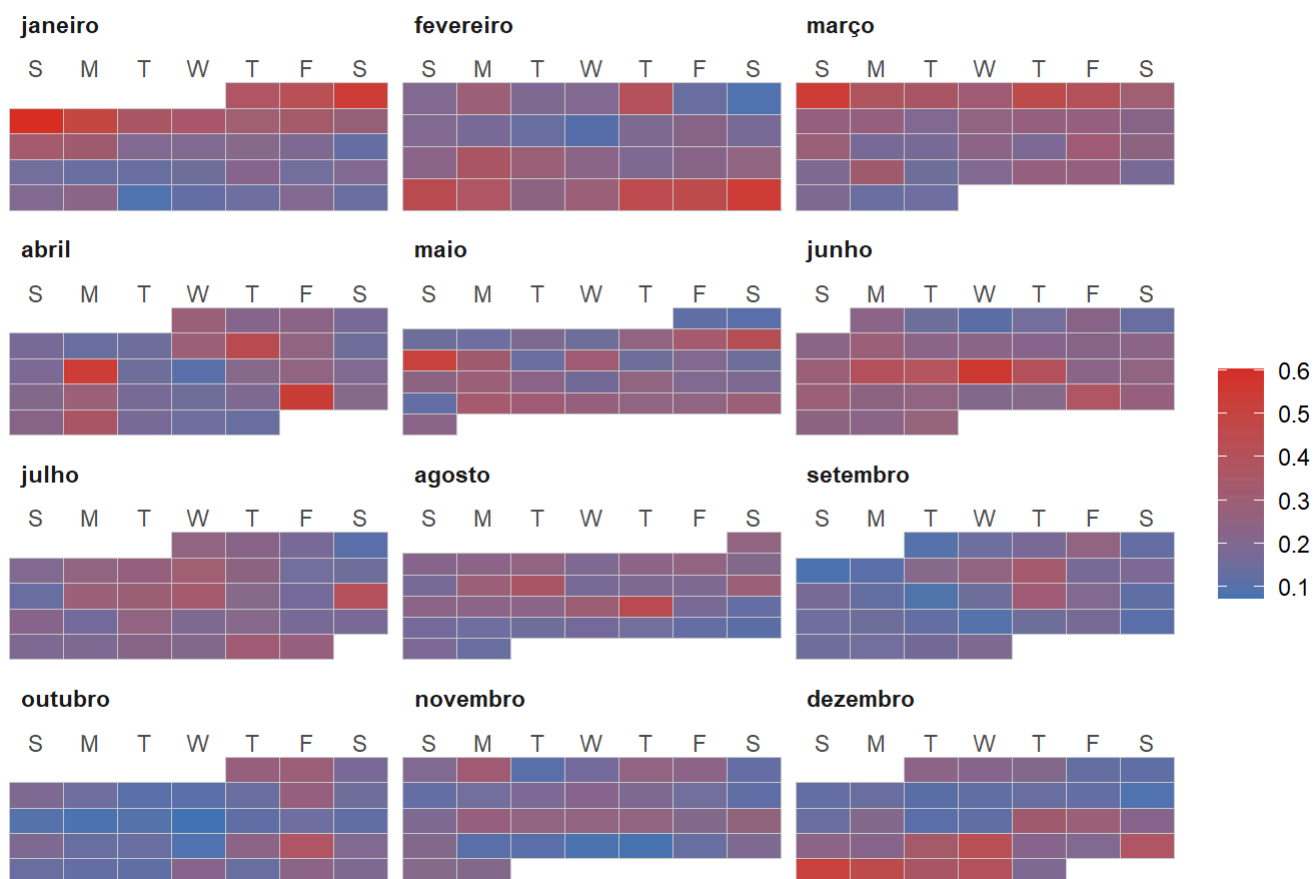
d. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`

```
cAA <- baseCalendario(stats_final, "AA")
cDL <- baseCalendario(stats_final, "DL")
cUA <- baseCalendario(stats_final, "UA")
cUS <- baseCalendario(stats_final, "US")
```

e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Título")`.

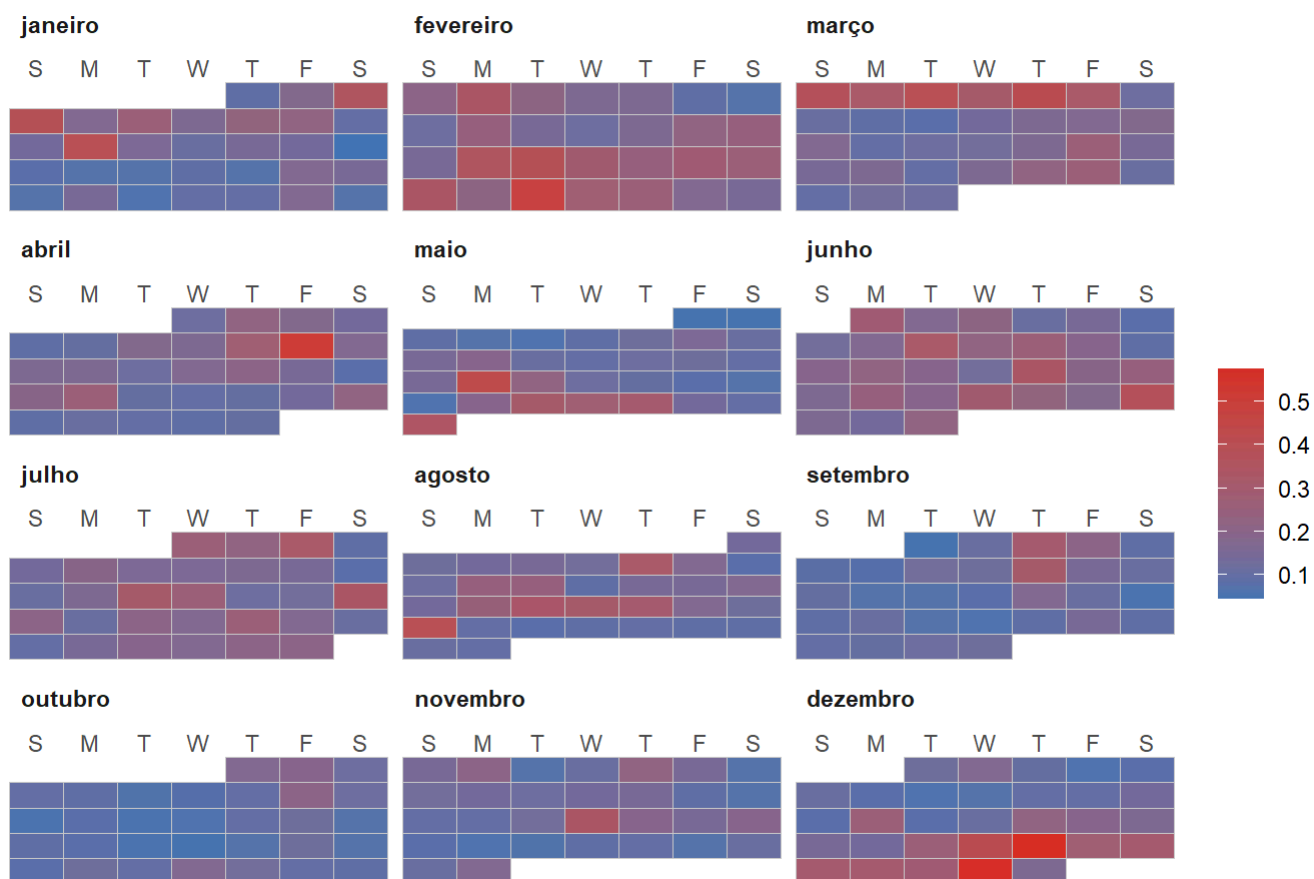
```
# Mapa de calor - American Airlines
cAA + pal + ggtitle("Percentual de Atrasos - American Airlines (AA)")
```

Percentual de Atrasos - American Airlines (AA)



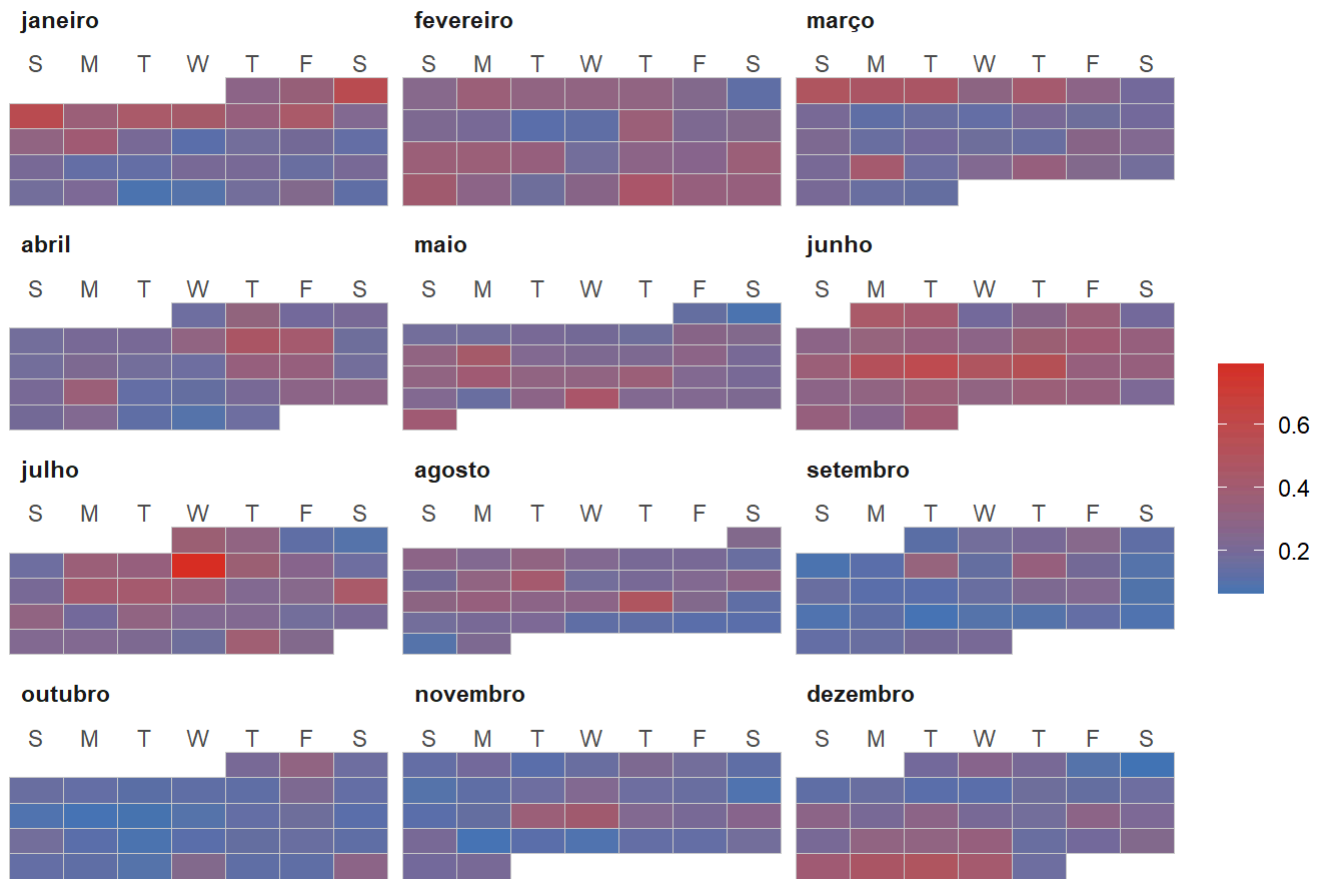
```
# Mapa de calor - Delta Air Lines
cDL + pal + ggtitle("Percentual de Atrasos - Delta Air Lines (DL)")
```

Percentual de Atrasos - Delta Air Lines (DL)



```
# Mapa de calor - United Airlines
CUA + pal + ggtitle("Percentual de Atrasos - United Airlines (UA)")
```

Percentual de Atrasos - United Airlines (UA)



```
# Mapa de calor - US Airways
```

```
cUS + pal + ggtitle("Percentual de Atrasos - US Airways (US)")
```

Percentual de Atrasos - US Airways (US)

